How much is there to gain from tuning an algebraic multigrid solver? A cross-library empirical study with cheap, honestly-priced matrix features

Chun-Yeol You
Department of Physics and Chemistry, DGIST
(Daegu Gyeongbuk Institute of Science and Technology),
Daegu 42988, Republic of Korea
cyyou@dgist.ac.kr

July 29, 2026

Abstract

Algebraic multigrid (AMG) is the workhorse preconditioner for large sparse linear systems, yet its performance depends strongly on internal parameters — coarsening scheme, smoother, and strength-of-connection threshold — that most users leave at their defaults. We ask two questions empirically. First, how much speed is left on the table by not tuning these *internal* knobs (as opposed to the coarser "which solver" choice studied previously)? Second, can a good configuration be predicted from features that are cheap to compute relative to the solve itself? Sweeping 88 parameter configurations over 150 matrices from the SuiteSparse collection with AMGCL (13,200 solves), we find a median oracle speedup of 2.4× over the default (interquartile top 6.4×, maximum 292×), with the best configuration varying across 14 distinct coarsening/smoother labels — so per-matrix selection, not a single better default, is required. A gradient-boosted predictor trained and evaluated with a strict leave-one-group-out protocol picks a configuration that solves 83% of held-out matrices and captures ∼43% of the achievable speedup, and *near-free structural features alone are as good as adding expensive spectral features that cost* 100–1000× *more*. Crucially, we cross-validate on HYPRE BoomerAMG and find that while tunability replicates qualitatively (median oracle 1.6×), its magnitude does not transfer between libraries (rank correlation 0.24) and is largely governed by default quality: the dramatic AMGCL speedups occur precisely where its default is weak and HYPRE's well-engineered default already wins. Intrinsic solvability, by contrast, is library-independent (hard-matrix Jaccard 0.76). All code, the sweep dataset, and the feature extractor are released for reproduction.

1 Introduction

Simulations across science and engineering reduce, at their core, to solving large sparse linear systems $Ax = b$ with millions of unknowns. Algebraic multigrid (AMG) [3, 4, 5] is among the most effective preconditioners for such systems and is embedded in every major solver framework [12, 13, 9, 11]. Its efficiency, however, hinges on a handful of internal parameters: the coarsening scheme that builds the multigrid hierarchy, the smoother that damps error on each level, and the strength-of-connection threshold that decides which couplings are "strong." These knobs interact, are problem-dependent, and are difficult to set a priori — so in practice they are left at library defaults that are rarely optimal.

A substantial literature studies *algorithm selection* for linear solvers [22, 24, 25, 27], but almost all of it operates at the level of "which solver / which preconditioner," treating an entire AMG method as a single categorical label. The interior of that label — the coarsening × smoother × threshold space that this paper targets — is comparatively unexplored, even though library documentation itself admits the defaults are problem-dependent. Two further methodological gaps recur. Feature-based selection studies seldom charge the cost of computing their features against the speedup those features buy; and predictor evaluation typically uses random cross-validation, which leaks sibling matrices from the same problem family across the train/test split and inflates reported accuracy.

This work is a systematic, AI-assisted empirical study in the spirit of recent performance-engineering case studies [1, 2]: we hold the measurement discipline fixed and vary only what we intend to measure. We make four contributions.

1. We quantify the oracle speedup available from tuning *internal* AMG parameters over 150 SuiteSparse matrices and 88 AMGCL configurations, and show it is large and requires per-matrix selection (Section 4).

2. We build a predictor from features tiered by cost, evaluate it with a strict leave-one-group-out protocol, and show that near-free structural features suffice — expensive spectral features add nothing despite costing 100–1000× more (Section 5).

3. We cross-validate on a second, independent AMG library (HYPRE BoomerAMG) and show that tunability replicates qualitatively but its magnitude is library-dependent and driven by default quality, whereas intrinsic solvability is library-independent (Section 6).

4. We release all tooling and the full sweep dataset for reproduction.

2 Related work

The algorithm-selection problem dates to Rice [22]; general surveys are given by Kotthoff [23]. For sparse linear solvers, Bhowmick et al. [24] framed solver selection as classification with feature-computation cost as an explicit concern. The most comprehensive study is Sood's [25], which labelled over 1,000 SuiteSparse matrices against 154 PETSc solver–preconditioner configurations and found that a small set of inexpensive matrix features nearly matches the full set. More recent work applies graph neural networks: Tang, Zhang and Chen [27] predict one of 33 PETSc solver+preconditioner classes, and a parallel line learns preconditioners directly [28, 29]. All of these choose *among* methods or configurations at a coarse granularity; none systematically tunes the interior parameters of a single AMG method, which is our focus.

On the AMG side, the classical Ruge–Stüben method [3] and smoothed aggregation [5] define the coarsening families we sweep; BoomerAMG [8, 9] and its parallel coarsening schemes [10] provide the second library for cross-validation, and AMGCL [11] the first. We use the SuiteSparse Matrix Collection [15] as the matrix source and Krylov methods [17, 18, 19, 16] as accelerators.

3 Methodology

Matrices. We draw 150 real, square matrices from the SuiteSparse collection [15] with $10^4 \le n \le 5 \times 10^5$ and at least three nonzeros per row on average, deliberately selecting *multiple* matrices from each of 43 problem groups so that a leave-one-group-out evaluation has within-family structure to

generalise across. Pure graph/network collections, on which AMG is not expected to work, are excluded.

Parameter grid. For AMGCL we cross three coarsening schemes (smoothed aggregation, plain aggregation, Ruge–Stüben) with four strength-threshold values, eight relaxation schemes (`spai0/1`, damped Jacobi, Gauss–Seidel, Chebyshev, ILU(0), ILU(k) for $k \in \{1, 2\}$), giving 88 configurations per matrix. SPD matrices are solved with conjugate gradients, others with BiCGStab. The strength-threshold path differs between coarsening families and is handled per-family.

Measurement discipline. Every configuration is timed as setup + solve to a relative residual of 10^{-8}, with a right-hand side $b = Ax^*$ for a fixed ramp $x_i^* = 1 + i/n$; the all-ones vector, used in a pilot, is degenerate for zero-row-sum matrices ($b = 0$) and was discarded. Each solve is repeated in process and the minimum reported, since scheduler noise can only add time. To avoid memory-bandwidth contention corrupting timings, we run one single-threaded solver process per compute node and distribute matrices across nodes; a contention probe confirmed that co-running even four solves inflates a 36 ms solve to 116 ms. Runs are capped at 30 s wall-clock; failures are recorded by type (diverged, stalled at max-iterations, timed out, errored). Experiments ran on the DGIST iREMB cluster.

Features and predictor. We extract features in three cost tiers: Tier 0 (structure: size, density, row-nonzero distribution, bandwidth, pattern symmetry, diagonal presence), Tier 1 (cheap value-based: diagonal dominance, value asymmetry, a Gershgorin bound, a few power-iteration steps), and Tier 2 (a sparse condition-number estimate). The predictor is a gradient-boosted classifier/regressor [31, 32] that, given matrix features and a one-hot configuration encoding, predicts whether a configuration will succeed and its log solve-time; per matrix it picks the fastest predicted-successful configuration. **All predictor numbers use leave-one-group-out cross-validation**: every matrix from a held-out SuiteSparse group is excluded from training, so no sibling leaks across the split.

4 How much is there to gain from tuning?

Figure 1 shows the per-matrix *oracle speedup*, the ratio of the default configuration's time to the best configuration's time, over the 41 matrices where both solved. The median is 2.4×, the upper quartile exceeds 6.4×, and the maximum reaches 292×; 21 of 41 matrices gain at least 2×. Beyond speed, tuning changes feasibility: on 31 matrices the default configuration fails outright while some configuration succeeds. Tuning is therefore not a marginal optimisation but often the difference between solving and not solving.

Critically, no single configuration dominates. Figure 2 counts how often each coarsening/relaxation label is the per-matrix winner: 14 distinct labels win across the matrices, and the AMGCL default (smoothed aggregation with `spai0`) is almost never among them. There is thus no "just change the default" fix; capturing the available speedup requires choosing per matrix.

5 Predicting a good configuration cheaply

Can the winning region be predicted without solving? Under leave-one-group-out evaluation, a per-(matrix, configuration) success classifier reaches accuracy 0.84 against a majority baseline of 0.76. Used end-to-end — picking one configuration per held-out matrix — the predictor selects a

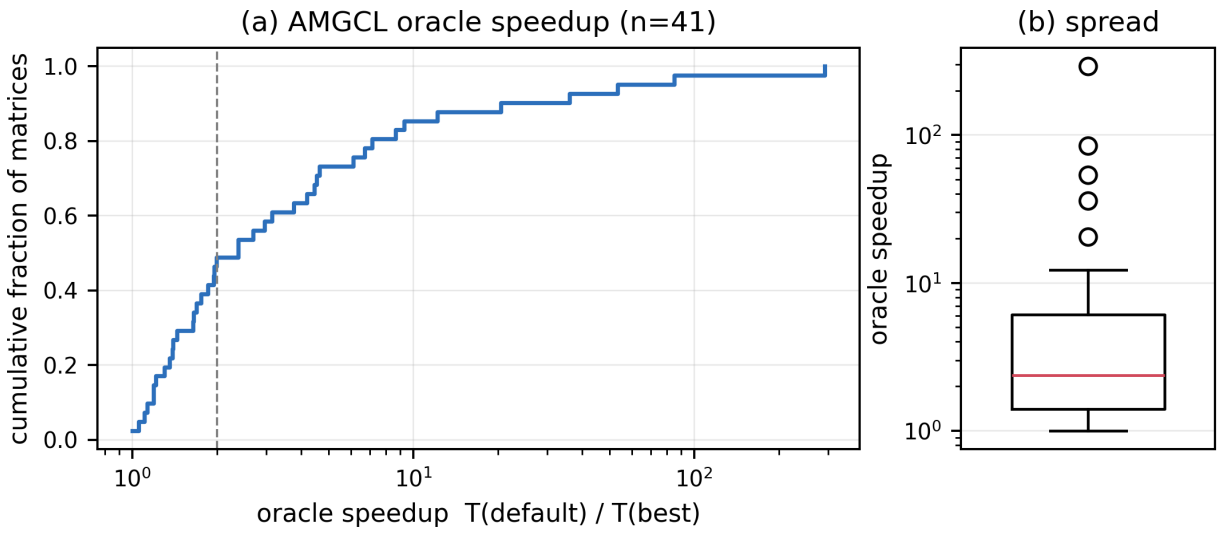

Figure 1: AMGCL oracle speedup over 41 matrices: (a) empirical CDF (log axis; dashed line at 2×), (b) distribution on a log scale. Median 2.4×, upper quartile 6.4×, maximum 292×.

configuration that actually solves 83% of the time and captures a median 43% of the oracle speedup (Figure 3), i.e. roughly a 1.45× median speedup over the default from features alone, while also steering away from the defaults that fail.

The most practically useful finding concerns feature cost. Tier 2 spectral features cost 100–1000× more than Tier 0/1 (Figure 4: median Tier 0 28 ms, Tier 1 +44 ms, Tier 2 +8.8 s per matrix, and the condition-number estimate frequently fails to converge). Yet adding Tier 1 to Tier 0 does not improve the predictor (43% → 40% captured), and Tier 2 is unusable at scale. Near-free structural features are sufficient. This directly addresses the feature-cost gap in prior work: for this task, the expensive features are not worth their price.

More data helps. To check that the predictor is not saturated by the 150-matrix training set, we doubled it to 300 matrices (a strict superset, preserving the multiple-per-group structure) and re-evaluated under the same leave-one-group-out protocol. The predictor improves rather than plateaus: the number of evaluable matrices rises from 41 to 66, the picked configuration solves 89% of held-out matrices (up from 83%), and the captured oracle speedup rises to a median 49% (from 43%), while the oracle distribution itself stays stable (median 2.3×, maximum 292×). Tier 0 features remain sufficient at the larger scale (49% vs. 37% for Tier 0+1). More labelled matrices therefore strengthen the signal, and the near-free feature finding is not an artifact of the smaller set.

6 Cross-library validation: does this generalise?

Are these findings specific to AMGCL, or a property of AMG? We repeat the sweep on HYPRE BoomerAMG [8] — an independent, widely-deployed library — over the same 150 matrices, with a matched 36-point grid (Falgout/PMIS/HMIS coarsening × four smoothers × three thresholds) and identical measurement discipline.

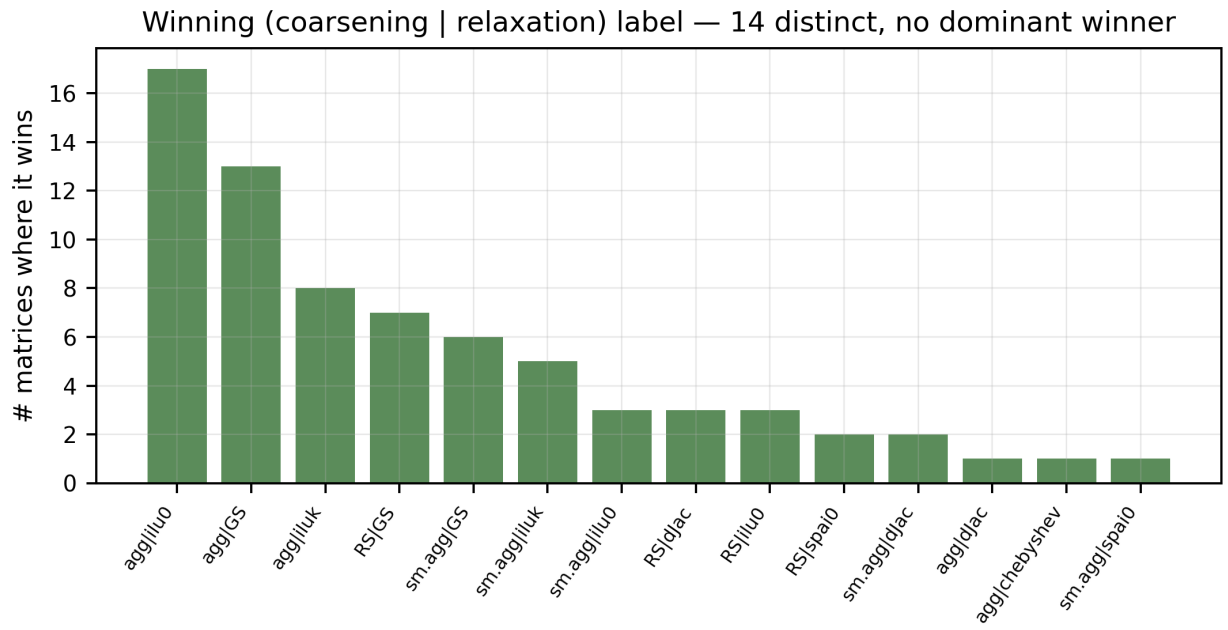

Figure 2: Frequency with which each (coarsening | relaxation) label is the per-matrix winner. The spread across 14 labels shows that per-matrix selection, not a single improved default, is what the speedup requires.

Tunability replicates, but modestly. HYPRE also exhibits a per-matrix oracle speedup (median 1.6×, upper quartile 2.1×, maximum 31×; 16 default-fails-tuned-succeeds matrices; 22 distinct winners). So "internal parameters matter" holds qualitatively in both libraries — but the magnitude is smaller in HYPRE.

Magnitude is default-driven, not intrinsic. Figure 5 plots per-matrix oracle speedups of the two libraries against each other: the rank correlation is only 0.24. A matrix that is very tunable in AMGCL is not necessarily tunable in HYPRE. Comparing the two libraries' *defaults* head-to-head explains why: HYPRE's default is faster on 26 of 36 shared matrices (median 0.70× the AMGCL-default time) and solves nine matrices the AMGCL default fails. Moreover, every one of the ten highest-AMGCL-oracle matrices (including the 292× case) is one where HYPRE's default already beats AMGCL's default. The dramatic AMGCL speedups are therefore largely a *weak-default artifact*: they measure how far AMGCL's default is from optimal on those matrices, not an intrinsic, library-independent tuning headroom.

Intrinsic difficulty is library-independent. By contrast, the set of matrices that no configuration solves overlaps strongly between libraries (Jaccard 0.76). Whether a problem is solvable by AMG at all is a property of the matrix; how much tuning helps is a property of the matrix–library pair.

A predictor confirms both directions. The same predictor approach applies to HYPRE: under leave-one-group-out it picks a configuration that solves 87% of held-out matrices and captures a median 47% of HYPRE's (modest) oracle — comparable to the AMGCL figures. More tellingly,

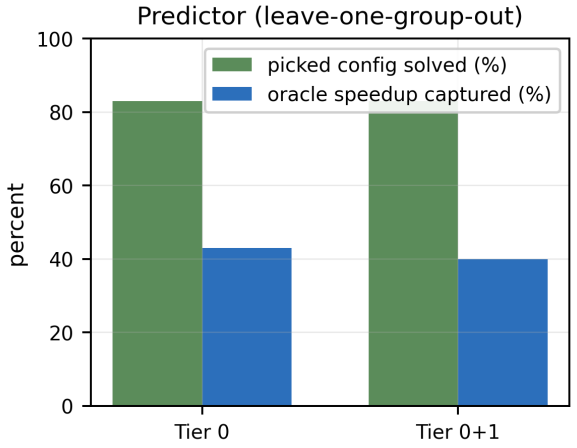

Figure 3: Predictor under leave-one-group-out: fraction of held-out matrices whose picked configuration solves, and fraction of oracle speedup captured. Tier 0 (near-free) matches Tier 0+1.

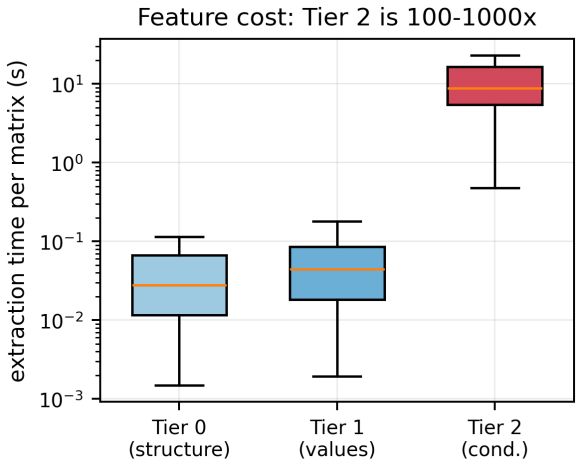

Figure 4: Feature-extraction time per matrix by tier (log scale). Tier 2 costs 100–1000× Tier 0/1 yet buys no predictive accuracy here.

transferring *matrix-level* models across libraries reproduces the two-part story. A "will any configuration solve this matrix?" classifier trained on one library and tested on the other beats its majority baseline by +0.25 to +0.33: intrinsic solvability transfers. But a regressor for the per-matrix oracle magnitude, trained on one library and tested on the other, predicts the target with rank correlation only ≈ 0.2: tuning benefit does not transfer. Difficulty is a library-independent property of the matrix; tunability is not.

7 Discussion

The picture that emerges is more nuanced than "tuning gives large speedups." Internal AMG parameter tuning yields real gains in both standard libraries, but their size is governed largely by how good the library's default already is, and does not transfer between libraries. This has a clear practical implication: automatic parameter selection is most valuable for libraries or regimes with weaker defaults, and its value should be reported *relative to a strong baseline* to avoid overstating it — a caveat our own AMGCL figures make concrete. At the same time, that a near-free feature set predicts useful configurations under an honest group-wise split is encouraging for building lightweight, always-on auto-tuning into solver libraries.

Limitations. The predictor captures $\sim 43\%$ of the oracle, not all of it; the study covers 150 matrices and two CPU AMG libraries; and cross-library absolute-time comparisons are confounded by implementation differences and are read only as trends. Extending to more matrices, to distributed/GPU backends, and to a HYPRE-native predictor with cross-library transfer are natural next steps.

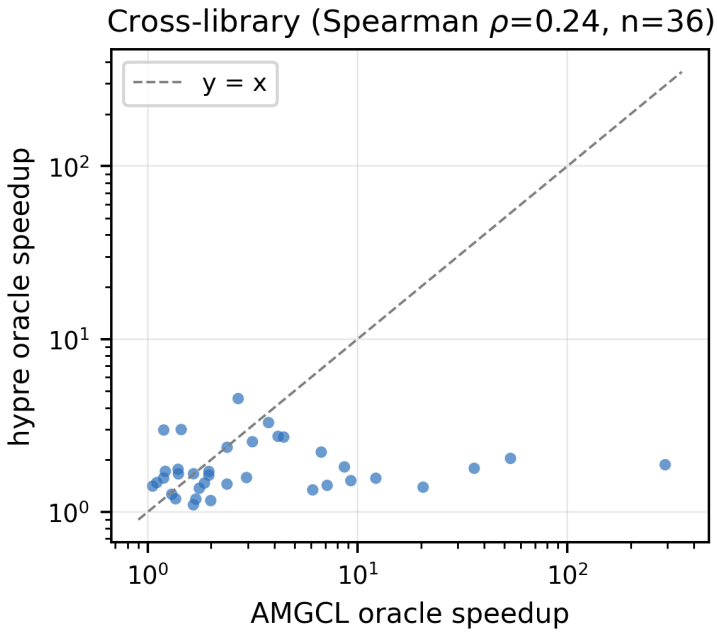

Figure 5: Per-matrix oracle speedup: AMGCL vs HYPRE (both log axes, $n = 36$). The weak rank correlation ($\rho = 0.24$) shows tuning benefit does not transfer between libraries; the largest AMGCL gains are where HYPRE's default already wins.

8 Conclusion

Tuning the internal parameters of algebraic multigrid — not merely choosing among solvers — offers a median 2.4× and occasionally order-of-magnitude speedup in AMGCL, requires per-matrix selection, and can be partly predicted from features that cost essentially nothing, with expensive spectral features adding no value. Cross-validation on HYPRE BoomerAMG shows the phenomenon is real but its magnitude is default-dependent and non-transferable, while intrinsic solvability is library-independent. We release all code and data so these claims can be reproduced and extended.

Acknowledgment

This work was supported by the National Research Foundation of Korea (NRF) (No. RS-2026-25502724) and the Strategic Research Program under the DGIST R&D Program (26-SR-01) of the Ministry of Science, ICT, and Future Planning.

Data and code availability

The sweep dataset (13,200 AMGCL and 5,400 HYPRE solves), the feature extractor, the predictor, and all figure-generation scripts are released in the project repository.

Generative AI disclosure

In accordance with the ACM Policy on Authorship, the author discloses the use of an AI coding assistant (Claude Code, Anthropic) in this work. The assistant helped implement the benchmark harness, the tiered feature extractor, and the gradient-boosted predictor; orchestrate the parameter sweeps and cross-library build on the compute cluster; and draft and revise this manuscript. All experimental design decisions, numerical results, and scientific claims were checked by the author against the released code and data. The AI system is not an author and bears no responsibility for the content; the author takes full responsibility for all material herein.

References

[1] C.-Y. You, *Optimization of MuMax3 by using Claude Code: A CUDAgraph-based case study in AI-assisted performance engineering*, J. Magn. **31** (2026) 204. doi:10.4283/JMAG.2026.31.2.204.

[2] C.-Y. You, *Variance-reduced trajectory unravelings for GPU noisy quantum-circuit simulation: Characterization and a Qiskit-Aer integration gap*, arXiv:2607.17678 (2026).

[3] J.W. Ruge, K. Stüben, *Algebraic multigrid*, in: Multigrid Methods, SIAM, 1987, pp. 73–130.

[4] K. Stüben, *A review of algebraic multigrid*, J. Comput. Appl. Math. **128** (2001) 281–309.

[5] P. Vaněk, J. Mandel, M. Brezina, *Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems*, Computing **56** (1996) 179–196.

[6] A. Brandt, S.F. McCormick, J. Ruge, *Algebraic multigrid (AMG) for sparse matrix equations*, in: Sparsity and Its Applications, Cambridge Univ. Press, 1985.

[7] J. Xu, L. Zikatanov, *Algebraic multigrid methods*, Acta Numerica **26** (2017) 591–721.

[8] V.E. Henson, U.M. Yang, *BoomerAMG: A parallel algebraic multigrid solver and preconditioner*, Appl. Numer. Math. **41** (2002) 155–177.

[9] R.D. Falgout, U.M. Yang, *hypre: A library of high performance preconditioners*, in: Computational Science – ICCS 2002, LNCS 2331, Springer, 2002, pp. 632–641.

[10] H. De Sterck, U.M. Yang, J.J. Heys, *Reducing complexity in parallel algebraic multigrid preconditioners*, SIAM J. Matrix Anal. Appl. **27** (2006) 1019–1039.

[11] D. Demidov, *AMGCL: An efficient, flexible, and extensible algebraic multigrid implementation*, Lobachevskii J. Math. **40** (2019) 535–546.

[12] S. Balay et al., *PETSc/TAO users manual*, Technical Report ANL-21/39, Argonne National Laboratory, 2023.

[13] M.A. Heroux et al., *An overview of the Trilinos project*, ACM Trans. Math. Softw. **31** (2005) 397–423.

[14] N. Bell, L.N. Olson, J. Schroder, *PyAMG: Algebraic multigrid solvers in Python*, J. Open Source Softw. **7** (2022) 4142.

[15] T.A. Davis, Y. Hu, *The University of Florida sparse matrix collection*, ACM Trans. Math. Softw. **38** (2011) 1:1–1:25.

[16] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.

[17] M.R. Hestenes, E. Stiefel, *Methods of conjugate gradients for solving linear systems*, J. Res. Natl. Bur. Stand. **49** (1952) 409–436.

[18] Y. Saad, M.H. Schultz, *GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems*, SIAM J. Sci. Stat. Comput. **7** (1986) 856–869.

[19] H.A. van der Vorst, *Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems*, SIAM J. Sci. Stat. Comput. **13** (1992) 631–644.

[20] L.N. Trefethen, D. Bau, *Numerical Linear Algebra*, SIAM, 1997.

[21] A. George, *Nested dissection of a regular finite element mesh*, SIAM J. Numer. Anal. **10** (1973) 345–363.

[22] J.R. Rice, *The algorithm selection problem*, Adv. Comput. **15** (1976) 65–118.

[23] L. Kotthoff, *Algorithm selection for combinatorial search problems: A survey*, in: Data Mining and Constraint Programming, LNCS 10101, Springer, 2016, pp. 149–190.

[24] S. Bhowmick, B. Toth, P. Raghavan, *Towards low-cost, high-accuracy classifiers for linear solver selection*, in: Computational Science – ICCS 2009, LNCS 5544, Springer, 2009, pp. 463–472.

[25] K. Sood, *Iterative Solver Selection Techniques for Sparse Linear Systems*, Ph.D. thesis, University of Oregon, 2019.

[26] V. Eijkhout, E. Fuentes, *Machine learning for multi-stage selection of numerical methods*, in: New Advances in Machine Learning, InTech, 2010.

[27] Z. Tang, H. Zhang, J. Chen, *Graph neural networks for selection of preconditioners and iterative solvers*, NeurIPS 2022 Workshop (GLFrontiers), 2022.

[28] J. Chen, *Graph neural preconditioners for iterative solutions of sparse linear systems*, arXiv:2406.00809 (2024).

[29] P. Häusner, O. Öktem, J. Sjölund, *Neural incomplete factorization: Learning preconditioners for the conjugate gradient method*, in: Northern Lights Deep Learning Conf. (NLDL), 2025.

[30] V. Trifonov et al., *Learning from linear algebra: A graph neural network approach to preconditioner design for conjugate gradient solvers*, arXiv:2405.15557 (2024).

[31] J.H. Friedman, *Greedy function approximation: A gradient boosting machine*, Ann. Statist. **29** (2001) 1189–1232.

[32] F. Pedregosa et al., *Scikit-learn: Machine learning in Python*, J. Mach. Learn. Res. **12** (2011) 2825–2830.

[33] R.D. Falgout, J.E. Jones, U.M. Yang, *The design and implementation of hypre, a library of parallel high performance preconditioners*, in: Numerical Solution of PDEs on Parallel Computers, Springer, 2006, pp. 267–294.